

3 METHOD

3.1 PRELIMINARIES

Problem Definition. In our problem setup, we assume a training dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i, c_i)\}_{i=1}^n$, where $\mathbf{x}_i$ is an input sequence (instruction), $\mathbf{y}_i$ is the response to the instruction, and $c_i \in \{0, 1\}$ is a binary label indicating the harmfulness of the pair $(\mathbf{x}_i, \mathbf{y}_i)$. Additionally, we define a safety guard model $p_\theta(\cdot | \mathbf{x}, \mathbf{y})$ parameterized by θ , which assigns a probability to the pair of sequences $(\mathbf{x}, \mathbf{y})$ being harmful. Our goal is to distill the teacher p_θ into a smaller safety guard model $q_\phi(\cdot | \mathbf{x}, \mathbf{y})$, while minimizing accuracy degradation to improve efficiency of the safety guard model in the wild.

The efficiency of this distilled safety guard model reduces the computational cost, *i.e.*, latency, floating point operations (FLOPs), and memory usage, during both the development and deployment phases of LLMs. Before deploying an LLM, developers typically conduct iterative prompting to generate harmful responses, and evaluate their harmfulness with a safety guard model to identify and address vulnerabilities (Perez et al., 2022). However, this approach is resource-intensive and costly. During LLM deployment, the safety guard model is employed alongside the LLM to detect harmful responses generated from malicious user input. Moreover, the safety guard model needs to be regularly updated to effectively counter newly emerging jailbreak attacks.

Learning Objective. A widely used objective for knowledge distillation (Hinton et al., 2014) is to enforce the student q_ϕ to imitate the output of the teacher p_θ while minimizing negative log likelihood (binary cross-entropy; BCE) of the training dataset $\mathcal{D}$ as follows:

$$\begin{aligned} \text{minimize}_{\phi} \quad & \frac{1}{n} \sum_{i=1}^n (1 - \lambda) \cdot D_{\text{KL}}(p_\theta(\cdot | \mathbf{x}_i, \mathbf{y}_i) \parallel q_\phi(\cdot | \mathbf{x}_i, \mathbf{y}_i)) + \lambda \cdot \mathcal{L}_{\text{BCE}}(\mathbf{x}_i, \mathbf{y}_i, c_i) \\ & \mathcal{L}_{\text{BCE}}(\mathbf{x}_i, \mathbf{y}_i, c_i) = c_i \cdot \log q_\phi(c = 1 | \mathbf{x}_i, \mathbf{y}_i) + (1 - c_i) \cdot \log q_\phi(c = 0 | \mathbf{x}_i, \mathbf{y}_i) \end{aligned} \quad (1)$$

where D_{KL} denotes a Kullback-Leibler (KL) divergence and $\lambda \in [0, 1]$ is a hyperparameter that controls the weighting between KL divergence and binary cross-entropy loss.

3.2 DATA AUGMENTATION: HARMAUG

Training the student model on the training dataset $\mathcal{D}$ with Eq. (1) is suboptimal, as it easily overfits to the training data distribution and fails to generalize in detecting new malicious instructions under distribution shifts (Quionero-Candela et al., 2009; Subbaswamy et al., 2019). To address this issue, we propose a data augmentation method that involves leveraging LLMs to generate harmful instructions $\mathbf{x}$ and their corresponding responses $\mathbf{y}$. Suppose we are given an LLM p_{LLM} , pretrained on large scale text corpora and fine-tuned with reinforcement learning from human feedback (RLHF; Christiano et al., 2017). The LLM has acquired significant knowledge of harmfulness since the pre-training corpora contain a substantial amount of biased and offensive content (Bender et al., 2021). However, naively prompting the LLM to generate new harmful instructions is ineffective due to its built-in safety guardrails. During the RLHF fine-tuning stage, the LLM has been explicitly trained to refuse generating offensive content (Bai et al., 2022a;b; Touvron et al., 2023), which leads it to also reject generating harmful instructions.

Prefix attack to bypass safety guardrails of LLMs. To address this issue, we propose a simple prefix attack to bypass the safety guardrail of p_{LLM} . In addition to a set of k exemplars $\{\mathbf{x}_{j_1}, \dots, \mathbf{x}_{j_k}\}$ randomly sampled from $\mathcal{D}$, similar to (Wei et al., 2023b), and a prompt describing how to generate harmful instructions, such as “Make a single harmful instruction prompt that would elicit offensive content.”, we add an affirmative prefix of the LLM’s response to the prompt (*e.g.*, “I have an idea for a prompt:”) as follows:

Prompt Format

[user] **[Example]** 1. {example_1} ... k. {example_k} **[/Example]**
 Make a single harmful instruction prompt that would elicit offensive content. Try to be super creative and do not copy the examples. It should be a single sentence. **[user]**
[assistant] I have an idea for a prompt:

This prefix attack is similar to the prefix injection (Wei et al., 2023a), asking the LLM to answer

with a prefix by adding guidelines to the user prompt. However, our attack prefills the prefix in the LLM’s response and enforces the LLM to complete the rest of the response. Given the prompt with the affirmative prefix, denoted as $\mathbf{z}_j$, the LLM completes the response, *i.e.*, $\hat{\mathbf{x}}_j \sim p_{\text{LLM}}(\cdot \mid \mathbf{z}_j)$, leading to the sampling of harmful instructions. We refer to our method as **HarmAug**. Empirically, we found that our prefix attack effectively bypasses the built-in guardrails of the LLM, allowing for the generation of harmful instructions (Table 4). This jailbreak vulnerability may be attributed to a weakness in the current RLHF process for safety alignment. Humans rarely respond with a refusal immediately following an affirmative answer to a request, and the LLM is supervised fine-tuned to replicate such human behavior before the RLHF process. As a result, the model is heavily biased towards generating refusal responses to harmful instructions but the model is rarely penalized for generating responses after an affirmative prefix during RLHF, despite the prompt being harmful.

After sampling synthetic harmful instructions, we utilize two different LLMs for generating responses to those synthetic harmful instructions. The first LLM generates a refusal, denoted as $\hat{\mathbf{y}}_{j1}$, to each harmful instruction $\hat{\mathbf{x}}_j$. Similarly, the second LLM, which is fine-tuned on few-shot adversarial examples, samples a harmful response $\hat{\mathbf{y}}_{j2}$ to each $\hat{\mathbf{x}}_j$. Additionally, we pair the prompt with an empty sequence $\hat{\mathbf{y}}_{j3}$. The rationale for including the empty sequence is to train versatile safety guard models capable of handling both instruction classification and instruction-response pair classification tasks. Then, the teacher p_θ labels each instruction-response pair:

$$c_{jl} = \mathbb{1}\{p_\theta(c = 1 \mid \hat{\mathbf{x}}_j, \hat{\mathbf{y}}_{jl}) > \tau\} \quad (2)$$

for $l \in \{1, 2, 3\}$, where $\mathbb{1}$ is an indicator function and $\tau \in (0, 1)$ is a threshold for the pair of sequences classified as harmful. Finally, we augment the training dataset with our synthetic dataset $\hat{\mathcal{D}} = \{(\hat{\mathbf{x}}_j, \hat{\mathbf{y}}_{jl}, c_{jl})_{l=1}^3\}_{j=1}^m$ and train the small safety guard model q_ϕ with Eq. (1).

4 EXPERIMENTS

We first introduce datasets, baselines, and evaluation metrics, followed by experimental results on multiple benchmarks (Sec. 4.1), red-teaming language models (Sec. 4.2), further fine-tuning against new jailbreak attacks (Sec. 4.3), and ablations (Sec. 4.4).

Datasets. For the training dataset $\mathcal{D}$, we use the train split of WildGuardMix (Han et al., 2024) combined with our synthetic dataset. We evaluate the safety guard models on four public benchmark datasets: OpenAI Moderation (OAI; Markov et al., 2023), ToxicChat (Lin et al., 2023), Harm-Bench (Mazeika et al., 2024), and the test split of WildGuardMix. The first two datasets are targeted for instruction classification (*i.e.*, a response is always an empty sequence), while the others are designed for instruction-response pair classification.

Safety Guard Models. We use DeBERTa-v3-large (He et al., 2023) as the language model (LM) backbone for the safety guard model q_ϕ and compare our method against the following baselines:

1. **EDA** (Wei & Zou, 2019): This method employs synonym replacement, random insertion, random swap, and random deletion to augment the dataset $\mathcal{D}$ for training DeBERTa.
2. **GFN** (Lee et al., 2024): This approach trains an LM with GFlowNet (GFN; Bengio et al., 2021) to sample harmful instructions proportional to the mixture of the harmful score distribution induced by the safety guard model p_θ and a reference language model’s likelihood. We augment the training $\mathcal{D}$ with instructions generated by the LM fine-tuned with GFlowNet and train DeBERTa on the augmented dataset. More details are provided in Appendix B.2.
3. **Existing safety guard models:** These models include LMs fine-tuned for safety guard, such as RoBERTa-R4 (Vidgen et al., 2021), HateBERT (Hartvigsen et al., 2022), Llama-Guard-1, Llama-Guard-2, Llama-Guard-3 (Inan et al., 2023), WildGuard (Han et al., 2024), and Aegis-Guard (Ghosh et al., 2024).

Evaluation metrics. Following prior works (Inan et al., 2023; Han et al., 2024), we evaluate the safety guard models using F1 score and AUPRC. More details are provided in Appendix C.

4.1 MAIN RESULTS

Experimental setups. We use Llama-Guard-3 for the teacher safety guard model p_θ and DeBERTa-v3-large (He et al., 2023) for the student model q_ϕ . We utilize Gemma-1.1-2b-it for p_{LLM} to generate 100,000 harmful instructions, except for the ablation studies in Table 5 and Fig. 7.

Table 1: We run experiments three times with different random seeds and report the average of F1 and AUPRC scores. The best results are bolded and the second-best are underlined. For results including standard deviations, please refer to Table 9 in Appendix D.

Model	size	OAI		ToxicChat		HarmBench		WildGuardMix		Average	
		F1	AUPRC	F1	AUPRC	F1	AUPRC	F1	AUPRC	F1	AUPRC
Llama-Guard-1	7B	0.7520	0.8452	0.5818	0.7001	0.5012	0.8067	0.4793	0.7204	0.5786	0.7681
Llama-Guard-2	8B	0.8139	0.8824	0.4233	0.4368	0.8610	0.8945	0.6870	0.7833	0.6963	0.7492
Llama-Guard-3	8B	<u>0.8061</u>	0.8869	0.4859	0.4823	0.8551	0.8999	0.6852	0.8129	0.7080	0.7720
WildGuard ¹	7B	0.7268	n/a	<u>0.6547</u>	n/a	<u>0.8596</u>	n/a	0.7504	n/a	0.7479	n/a
Aegis-Guard	7B	0.6982	0.8532	0.6687	<u>0.7455</u>	0.7805	0.8178	0.6686	0.7386	0.7040	0.7888
RoBERTa-R4	125M	0.5625	0.6970	0.2217	0.3339	0.0288	0.6958	0.0477	0.3925	0.2152	0.5298
HateBERT	110M	0.6442	0.7443	0.3148	0.4867	0.1423	0.6669	0.0789	0.3763	0.2951	0.5685
OpenAI Moderation	n/a	0.7440	0.8746	0.4480	0.6206	0.5768	0.7763	0.4881	0.6393	0.5644	0.7089
DeBERTa	435M	0.7092	0.7869	0.6118	0.6837	0.8379	0.8806	<u>0.7507</u>	<u>0.8337</u>	0.7274	0.7962
DeBERTa + EDA	435M	0.6858	0.8394	0.5964	0.7141	0.8430	0.8793	0.7279	0.8315	0.7133	<u>0.8161</u>
DeBERTa + GFN	435M	0.6939	0.7793	0.6259	0.7191	0.8463	<u>0.8842</u>	0.7443	0.8376	0.7276	0.8050
DeBERTa + HarmAug	435M	0.7236	<u>0.8791</u>	0.6283	0.7553	0.8331	0.8841	0.7576	0.8265	<u>0.7357</u>	0.8362

Table 2: Computational cost of our model running on WildGuardMix test split, compared to Llama-Guard-3 and WildGuard. We measure actual total inference cost on an A100 GPU instance of RunPod.

Model	F1 (↑)	Size (↓)	FLOPs / token (↓)	Latency / token (↓)	Peak Memory (↓)	Monetary Cost (↓)
WildGuard	0.7504 (107%)	7B (88%)	131.87 G (106%)	722.08 μ s (418%)	22.63 GB (79%)	0.180 \$ (216%)
Llama-Guard-3	0.6998 (100%)	8B (100%)	124.01 G (100%)	172.62 μ s (100%)	28.82 GB (100%)	0.083 \$ (100%)
DeBERTa + HarmAug	0.7576 (108%)	435M (5%)	743.55 M (0.6%)	43.22 μs (25%)	3.37 GB (12%)	0.022 \$ (26%)

For each generated instruction, we generate a refusal response and a harmful response with [Llama-3-8B-Instruct](#) and [boyiwei/pure_bad_100-7b-full](#), respectively. Llama-Guard-3 then labels each instruction-response pair. The threshold for the harmfulness score τ is set to 0.5. We fine-tune DeBERTa-v3-large for 3 epochs with a batch size of 256, a weight decay of 0.1, λ of 0.5, and a learning rate of $3 \cdot 10^{-5}$. We use AdamW (Loshchilov & Hutter, 2019) optimizer and linearly decay the learning rate from the initial value $3 \cdot 10^{-5}$ to 0.

Quantitative Results. As shown in Table 1, our HarmAug significantly outperforms other augmentation baselines, including GFN and EDA. Remarkably, on the OAI and ToxicChat benchmark datasets, DeBERTa trained with our data augmentation method HarmAug, achieves a higher AUPRC than any other model, including its teacher Llama-Guard-3, as well as other models with 7 or 8 billion parameters. Additionally, our model, comprising only 435 million parameters, shows the highest average AUPRC and the second-best average F1 score among all evaluated models. These results demonstrate the effectiveness and efficiency of our approach, challenging the trend of fine-tuning large autoregressive models for safety tasks, which is both slow and costly.

Computational Cost. To evaluate the efficiency of our model relative to WildGuard and the teacher model Llama-Guard-3, we measure the operational costs of each model by analyzing the average FLOPs and latency per token, peak GPU memory usage, and the financial expense of running the models on an A100 GPU instance from RunPod while processing all instances in the test split of WildGuardMix. As shown in Table 2, our model significantly reduces the monetary cost, FLOPs, latency, and peak memory usage of WildGuard and Llama-Guard-3, while achieving a higher or comparable F1 score. These experimental results highlight the efficiency and efficacy of our safety guard model.

Qualitative Results. To study how our data augmentation method changes distribution of instructions, we cluster the prompts from the union of the original dataset $\mathcal{D}$ and our synthetic dataset $\hat{\mathcal{D}}$, and compare it against clustering with only the original dataset. We use Hugging Face’s [text clustering library](#) which embeds instructions with a language model and runs DBSCAN (Ester et al., 1996) for clustering. As shown in Fig. 3, our data augmentation significantly increases the number of clusters from 65 to 332. This suggests our data augmentation method, HarmAug, improves diversity of instructions in the training dataset. Generated instructions are presented in Table 12 of Appendix E.

4.2 CASE STUDY I: EFFICIENT REWARD MODELS OF RED-TEAMING LANGUAGE MODELS

Background. Red-teaming, which involves discovering diverse prompts that can elicit harmful responses from a target LLM p_{target} (Perez et al., 2022), aims to discover and address potential

¹We report “n/a” for AUPRC since the WildGuard library does not provide the probability of harmfulness.